

chip-flow

One engine that drives three Claude Code skills through an open-tool ASIC flow, each block checked against a corpus of reference designs with faults planted in it on purpose.

chip-flow is hwde's engine ported to chip design: the same state file, gate runner and violation-clustering script, aimed at Verilog and analog netlists instead of KiCad boards. Three skills sit on top of it. `/vde` takes a digital block from spec to a hardened, timed netlist. `/ade` takes an analog block from spec to a DRC- and LVS-clean, parasitic-extracted layout. `/msde` splits a mixed-signal spec into a digital and an analog side, runs each as a nested `/vde` or `/ade` run, then joins them and checks the join.

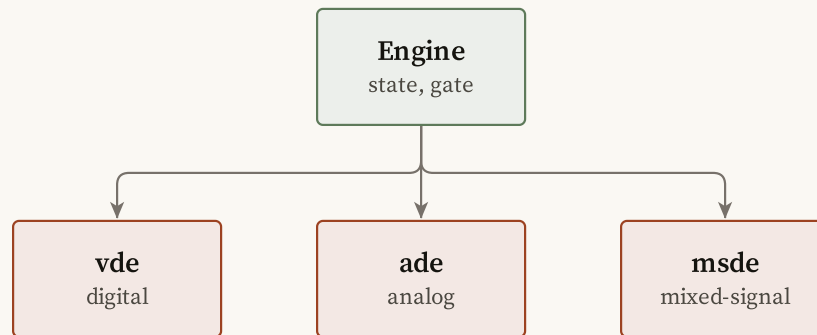
The problem it answers is the same one hwde answers for boards: a check that always passes is worthless, so every gate is proven against a corpus design with a specific fault planted in it, and the gate has to be the thing that catches that fault, not a coincidence.

How one block is built

1. **Spec and testbench.** A spec is written before any RTL or netlist, and the testbench is written before the design it will test.
2. **Design and its own gates.** Digital: lint, simulation, a holdout test set the design-writer never saw, a mutation test on the testbench itself, formal proof, coverage. Analog: topology and sizing checks, then simulation across corners.
3. **Human hold 1.** A checkpoint before anything expensive runs.
4. **Harden or lay out.** Digital synthesizes and places-and-routes through LibreLane and OpenSTA, as a background job because it can run over an hour, then timing, DRC, LVS and gate-level simulation against the hardened netlist. Analog draws the layout as Python code against the GF180 PDK, then DRC, LVS and a post-layout, parasitic-extracted resimulation.
5. **Release, human hold 2.** `attest.py` walks every gate that applies to the block, confirms each has a fresh pass on the files as they are now, and writes `reports/checks.json`. A gate that

could not run is refused, not skipped quietly. That file is what a person signs off on, not an agent's summary of it.

FIGURE 1 *the parts*



One engine underneath; each skill owns its own gates and its own corpus of faults.

The hard parts

Testing the test, not just the design

A testbench that asserts nothing will pass any design, including a broken one. Mutation testing makes a fixed set of mutants of the RTL (a removed reset, a stuck output, an inverted condition) and requires the visible tests to kill at least 90% of them, with no survivor in a must-kill class. A mutant proven logically equivalent to the original doesn't count against the score; anything else is a real gap. Analog has the matching idea, `bench_strength`: every planted device fault (a doubled size, a removed connection, a halved bias) has to push some measured quantity out of the bench's bounds.

A held-out test is only as strong as the isolation around it

Inside one run, the design-writer and fixer agents never see the files under `holdout/`, so they can't write to the test instead of the spec; but that's discipline, not a hard wall. The corpus's

own held-out tests live outside any workspace and only run after the agent's run has ended, which is the strong form of the same idea, and the one meant to be trusted.

Layout as code, judged by tools that never touch the GDS

The analog layout generator is a Python script, not a drawing; `layout_gen.py` runs it and writes the GDS. Magic, KLayout and netgen only ever judge the result. A DRC finding becomes a work order for the generator, not an edit to the GDS itself, so the same script can be rerun and reproduced. Parasitics come from magic's own extractor first, checked a second way through `klayout_pex`.

Mixed-signal at the seam

/msde's split gate checks the interface spec signal by signal, direction, level, domain and load, on both sides, before either is built. The two sides run and gate independently, then join through a cosim bench where ngspice's `d_cosim` loads the Icarus-compiled digital side inside the analog simulation. That bridge is thinly documented upstream, so it's proven first on a two-inverter case; a lock-step driver of ngspice's shared library is the fallback if it doesn't hold up.

What actually cleared the gates, and what didn't

A current mirror and a comparator have each gone from spec through DRC, LVS and post-layout, parasitic-extracted simulation to a signed release record; both release checks exist on disk. One mixed-signal example, a sensor whose output gets counted, reached every gate green on the merged engine, but the ladder that scores runs doesn't count it: two of the files behind that result are `mutant_rulings.yaml` entries a person edited by hand, and the ladder's rule is that a hand-edited run doesn't score. The larger digital designs don't clear their gates yet: the UART's mutation kill rate sits at 0.79 against a 0.90 bar, and the SPI FIFO's formal proof times out before it can close. Nothing here has been designed as a chip that could be submitted anywhere; submitting to a shuttle is a person's call that hasn't come up yet.

What it doesn't do

- No chip has been taped out, and nothing has gone from Verilog to a finished GDS the way that phrase is usually meant. The claims above are specific about how far each block got.
- gLayout, the layout library the analog side is built on, is unmaintained upstream and pinned against an older version of gdsfactory than the tool image ships; a generator written directly against the PDK's primitive cells is the fallback if that breaks. Every tool runs through a pinned Docker image and a version check, but the cosim bridge and the mutation gate are both new enough that their numbers are first measurements, not settled ones.

From the public repo <https://github.com/ihsan-sa/chip-flow>: docs/design.md, evals/ladder.md, engine/scripts/state.py, corpus/vde, corpus/ade and docs/runs/ade-mirror, docs/runs/ade-comparator, read 26 September 2026.